



Instituto Tecnológico De Costa Rica

Escuela de Ingeniería en Computación

IC-6821 — Diseño de Software

Estudiantes:

Luis Daniel Barboza Alfaro 2024211470

Jose Mario Castro Cruz 2022437423

Alexander Murillo Sancho

Proyecto 1: Requerimientos, Modelado e Implementación de
Backend

Sistema de Gestión Clínica

2026

Tabla de contenido

Tabla de contenido.....	2
Descripción del Sistema.....	3
Especificación de Casos de Uso.....	3
Modelo del Dominio.....	7
Tablas en la Base de Datos.....	8
Decisiones de Diseño.....	8
Repositorio de Código Fuente.....	8

Descripción del Sistema

El Sistema de Gestión Clínica es una aplicación web de backend diseñada para digitalizar y centralizar la operación de una clínica médica. Permite gestionar el ciclo completo de atención al paciente: desde la creación del perfil del paciente y la asignación de citas médicas, hasta el registro del historial clínico y la emisión de recetas.

El sistema está orientado a dar soporte a los flujos de trabajo del personal médico (doctores) y administrativo, brindando una API REST robusta que puede ser consumida por cualquier cliente (aplicación web, móvil u otro sistema).

Especificación de Casos de Uso

Los siguientes casos de uso describen las interacciones principales entre los actores del sistema y la aplicación. Se utiliza la notación basada en el Capítulo 9 de UML Distilled (Martin Fowler), priorizando la claridad del flujo sobre el apego estricto al formato.

Actor principal del sistema: Doctor / Personal Administrativo.

Caso de Uso: CU-01: Registrar Paciente	
Actor	Doctor / Personal Administrativo
Descripción	El sistema permite registrar un nuevo paciente con sus datos personales básicos, generando un registro en el sistema y un historial médico inicial vacío.
Precondición	El paciente no está registrado en el sistema.
Flujo principal	1. El actor envía los datos del paciente (nombre, apellido, fecha de nacimiento, sexo, dirección, teléfono, correo) vía API. 2. El sistema valida que los campos obligatorios estén presentes. 3. El sistema persiste el nuevo paciente en la base de datos. 4. El sistema crea automáticamente un historial médico vacío asociado al paciente. 5. El sistema retorna los datos del paciente creado con su ID asignado.
Postcondición	El paciente queda registrado en el sistema con un historial médico asociado.

Caso de Uso: CU-02: Agendar Cita Médica	
Actor	Doctor / Personal Administrativo
Descripción	El sistema permite agendar una cita médica para un paciente con un doctor en una fecha y hora específica, validando la disponibilidad del horario.
Precondición	El paciente, el doctor y la especialidad deben existir en el sistema.
Flujo principal	1. El actor envía los datos de la cita (doctorId, pacienteId, especialidadId, fecha, hora, motivo) vía API. 2. El sistema verifica que el horario solicitado no esté ocupado por otra cita activa (PROGRAMADA o ATENDIDA). 3. Si el horario está libre, el sistema crea la cita con estado PROGRAMADA. 4. El sistema retorna la cita creada con su ID. Flujo alternativo: Si el horario está ocupado, el sistema retorna un error 409 CONFLICT con el motivo.
Postcondición	La cita queda registrada con estado PROGRAMADA.

Caso de Uso: CU-03: Registrar Atención de Cita	
Actor	Doctor
Descripción	El sistema permite registrar la atención de una cita médica, creando el historial médico correspondiente con diagnóstico, tratamiento y recetas.
Precondición	La cita debe existir y tener estado PROGRAMADA. No debe tener un historial ya registrado.
Flujo principal	1. El actor envía los datos de la atención (citaId, diagnóstico, tratamiento, fecha, recetas opcionales) vía API. 2. El sistema verifica que la cita exista y esté en estado PROGRAMADA. 3. El sistema verifica que la cita no tenga ya un historial asociado. 4. El sistema crea el historial médico con los datos provistos. 5. El sistema persiste las recetas asociadas al historial (si se proporcionaron). 6. El sistema cambia el estado de la cita a ATENDIDA. 7. El sistema retorna el historial médico creado con sus recetas.
Postcondición	El historial médico queda registrado y la cita cambia a estado ATENDIDA.

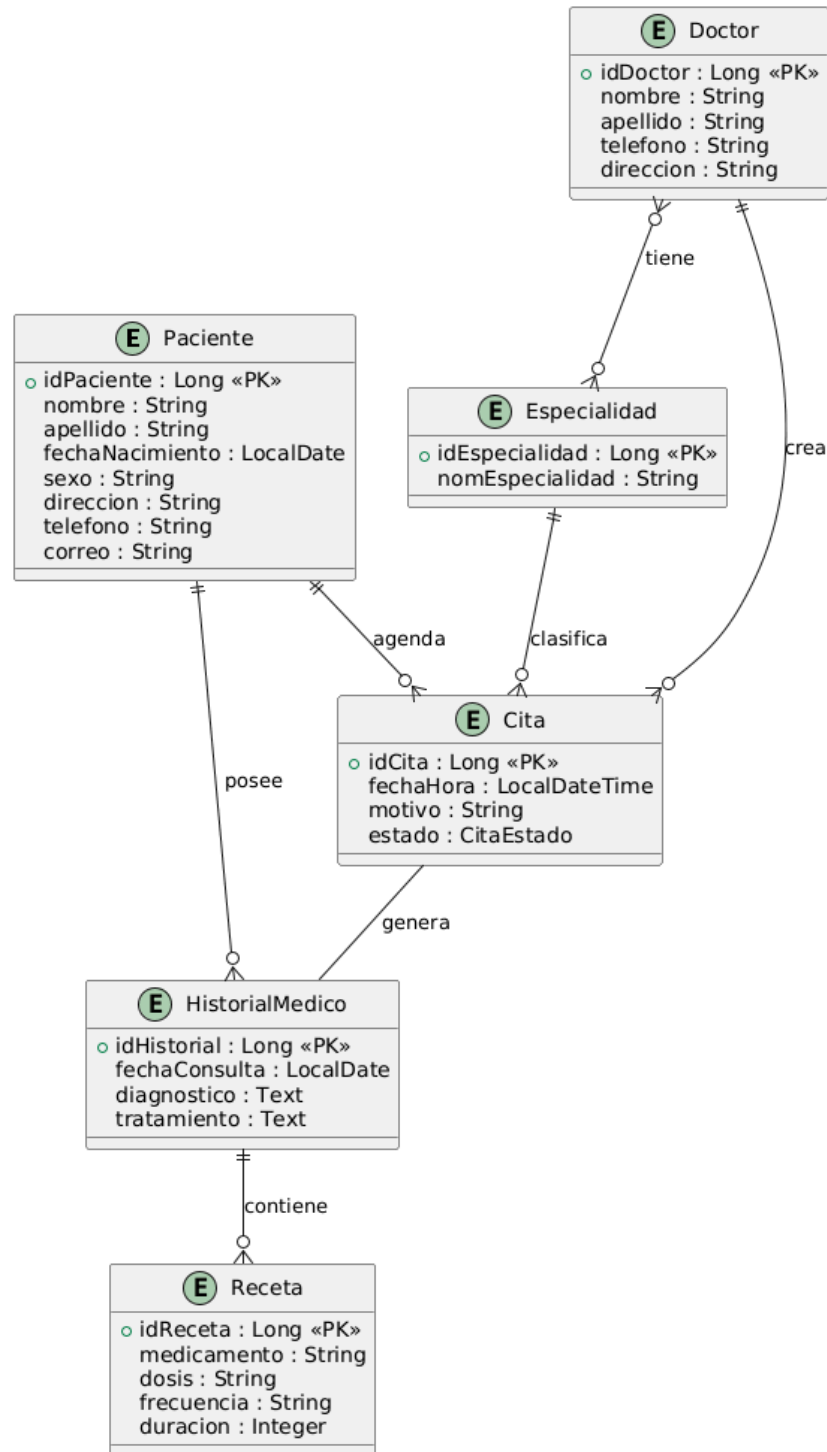
Caso de Uso: CU-04: Cancelar Cita	
Actor	Doctor / Personal Administrativo
Descripción	El sistema permite cancelar una cita médica existente, liberando el horario para futuras asignaciones.
Precondición	La cita debe existir en el sistema.
Flujo principal	1. El actor envía el ID de la cita a cancelar vía API (PATCH /citas/{id}/cancelar). 2. El sistema localiza la cita por su ID. 3. El sistema cambia el estado de la cita a CANCELADA. 4. El sistema retorna la cita con su nuevo estado.
Postcondición	La cita queda con estado CANCELADA y el horario queda disponible.

Caso de Uso: CU-05: Consultar Historial Médico de Paciente	
Actor	Doctor
Descripción	El sistema permite consultar todos los historiales médicos de un paciente, ordenados del más reciente al más antiguo.
Precondición	El paciente debe existir en el sistema.
Flujo principal	1. El actor envía el ID del paciente vía API (GET /historiales/por-paciente/{id}). 2. El sistema busca todos los historiales asociados a ese paciente. 3. El sistema retorna la lista de historiales ordenados por fecha descendente, incluyendo las recetas de cada uno.
Postcondición	El actor recibe el listado completo del historial clínico del paciente.

Caso de Uso: CU-06: Gestionar Doctores y Especialidades	
Actor	Administrador
Descripción	El sistema permite crear, consultar, actualizar y eliminar doctores, así como asignarles especialidades médicas.
Precondición	Ninguna.
Flujo principal	1. Para crear un doctor: el actor envía nombre, apellido, teléfono y dirección (POST /api/doctores). 2. Para asignar especialidades: el actor envía la lista de IDs de especialidad al doctor (PUT /api/doctores/{id}/especialidades). 3. Para crear una especialidad nueva: el actor envía el nombre (POST /api/especialidades). 4. El sistema valida que no exista ya una especialidad con el mismo nombre. 5. El sistema persiste los cambios y retorna los datos actualizados.
Postcondición	El doctor y/o especialidad quedan registrados o actualizados en el sistema.

Modelo del Dominio

El modelo del dominio representa las entidades principales del sistema y sus relaciones, siguiendo los lineamientos de Rosenberg & Stephens en Use Case Driven Object Modeling with UML. Se identificaron las siguientes entidades con sus atributos y relaciones.



Tablas en la Base de Datos

El modelo relacional cuenta con 8 tablas:

- paciente: Datos personales del paciente
- doctor: Datos del médico
- especialidad: Catálogo de especialidades médicas
- doc_especialidad: Tabla intermedia para la relación Doctor-Especialidad
- cita: Registro de citas médicas con su estado
- historial_medico: Registro de atenciones médicas
- receta: Medicamentos prescritos en cada atención

Decisiones de Diseño

Se eligió implementar una API REST en lugar de GraphQL debido a su simplicidad, facilidad de mantenimiento y adecuada adaptación al sistema. Las entidades como Doctor, Paciente, Cita, Historial y Receta se representan naturalmente como recursos con operaciones CRUD bien definidas, lo que hace que REST sea una opción clara y estructurada. Además, Spring Boot ofrece soporte nativo para REST sin necesidad de configuraciones adicionales, a diferencia de GraphQL que requiere más complejidad. Dado que los flujos del sistema son predecibles y no necesitan consultas dinámicas, REST resulta suficiente y, al ser un estándar ampliamente adoptado, facilita la integración con distintos tipos de clientes.

Repositorio de Código Fuente

El código fuente del proyecto se encuentra disponible en el siguiente repositorio de GitHub:

URL del repositorio: <https://github.com/CheuzEx/proyectodisenodesoftware1>